

Table of Contents

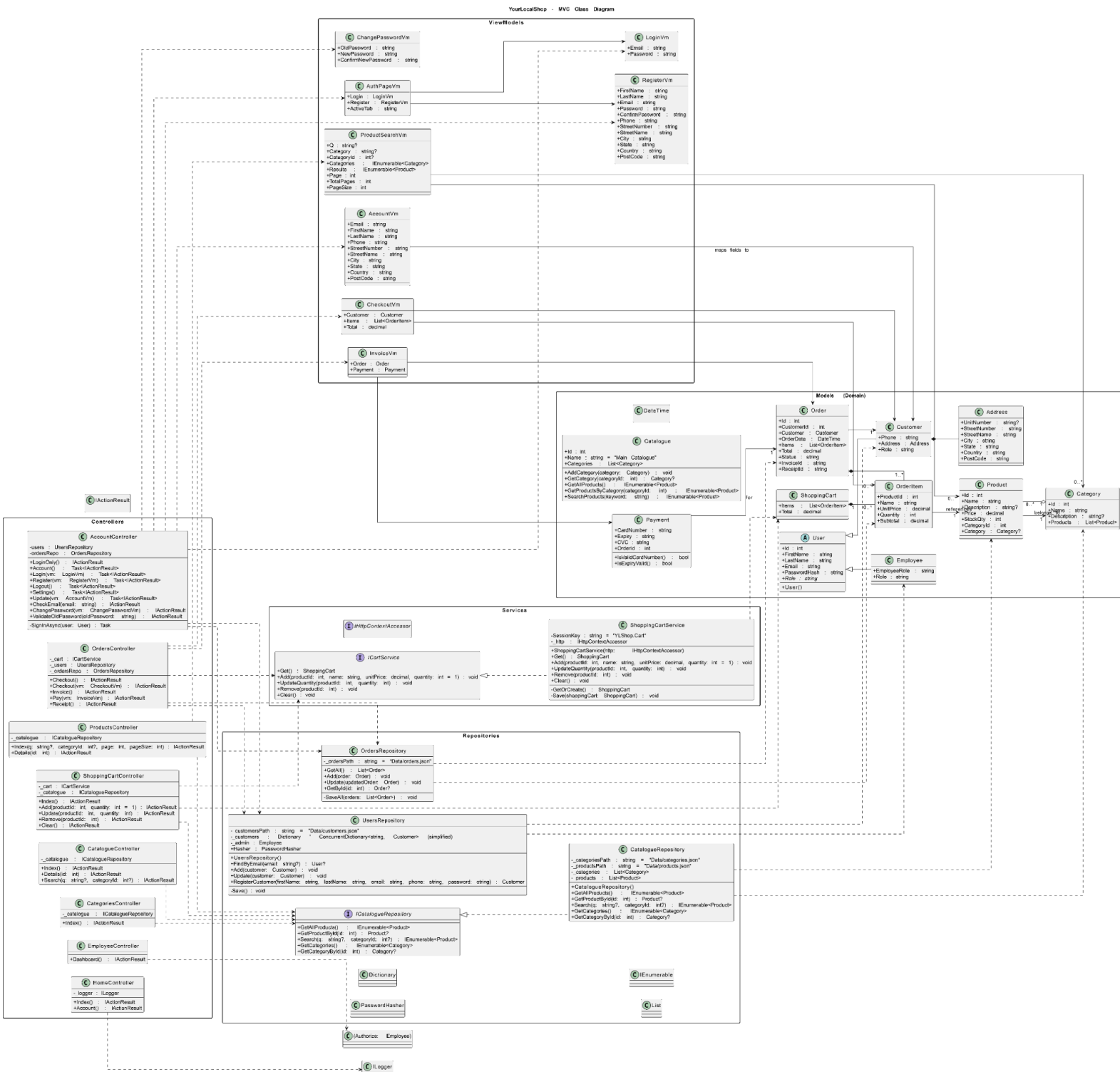
1. Detailed Design and Reflection.....	3
1.1. Class Diagram.....	3
1.1.1. Class Diagram Description and Justification.....	4
1.2. Justification of Changes and Non-Changes.....	5
1.2.1. Overall Changes/Non-Changes (at Class Level).....	5
1.2.2. Changes/Non-Changes to Responsibilities and Collaborators.....	8
1.2.3. Changes/Non-Changes to Dynamic Aspects (Bootstrap Process, Interaction Scenarios).....	12
2. Assignment 2 Design Discussion.....	14
2.1. Good Aspects of Assignment 2 Design (Original Design).....	14
2.2. Missing from Original Design.....	14
2.3. Flawed Aspects of Original Design.....	14
2.4. Level of Interpretation Required.....	15
3. Lessons Learnt.....	15
4. Architectural Styles.....	16
4.1. Layered Architecture.....	16
4.2. Model-View-Controller (MVC).....	17
4.3. Repository Pattern.....	17
4.4. Client-Server Architecture.....	17
4.5. Constraints.....	18
4.6. Diagram Reference.....	18
5. Implementation.....	19
5.1. Chosen Language, Coding Standard Followed and Data Storage.....	19
5.1.1. Chosen language.....	19
5.1.2. Coding Standard Followed.....	20
5.1.3. Data storage.....	20
5.2 Platform Used for Development and Testing of the Implementations.....	21
5.2.1. Platforms Used.....	21
5.2.2. Deployment and Execution.....	21
5.2.3. Application Startup.....	21
5.3. Four Areas of Business Operations Implemented.....	22
5.3.1. Create Customer Account, Login and Logout.....	22
5.3.2. Browse Products, Search Products and Filter Products by Category.....	23
5.3.3. Shopping Cart: Add Products to Cart, Change Product Quantity in Cart and Delete Product from Cart.....	23
5.3.4. Place Order: Checkout, Payment and Order History.....	24
6. References.....	25
7. Appendix.....	26

7.1. Appendix 2: Assignment 2 Link.....	26
---	----

1. Detailed Design and Reflection

1.1. Class Diagram

Figure 1: UML Class Diagram



Note: A high-resolution PDF version of the UML class diagram is available for download [here](#).

Due to the size and level of detail of the UML Class Diagram ([Figure 1](#)), a link to a viewable and downloadable PDF version is provided in the [Note](#) above or [here](#). The PDF allows for zooming in to view individual elements clearly and can also be downloaded for closer inspection if preferred.

1.1.1. Class Diagram Description and Justification

The system follows ASP.NET Core MVC with a clear split between Controllers (request handling), a thin Service layer (shopping cart operations), Repositories (file-backed persistence and lookups), Domain Models (Order, Customer, Product, etc.), and View Models (strongly-typed page models). Controllers depend on services/repos via DI; services/repos depend on domain types; views bind to VMs.

Key groupings:

- **Controllers:** HomeController, AccountController, EmployeeController, CatalogueController, CategoriesController, ProductsController, ShoppingCartController, OrdersController. They orchestrate flows like login/register, browse/search/filter, cart, checkout, invoice/payment, and order history.
- **Services:** ICartService/ShoppingCartService encapsulates cart mutations and session persistence, keeping controllers slim.
- **Repositories:** ICatalogueRepository/CatalogueRepository (read products/categories from JSON), UsersRepository (customers + in-memory admin), OrdersRepository (orders in JSON). Repos isolate file I/O and serialization.
- **Domain Models:** User (abstract), Customer, Employee, Address, Category, Product, Order, OrderItem, ShoppingCart, Payment, Catalogue. **Associations:** Customer has-a Address; Category aggregates many Product; Order composes many OrderItem and references a Customer; ShoppingCart composes many OrderItem; Payment is validated and tied to an Order.
- **View Models:** AuthPageVm, LoginVm, RegisterVm, AccountVm, ChangePasswordVm, ProductSearchVm, CheckoutVm, InvoiceVm drive forms and pages with validation and display-only props.

Justification:

The class diagram is structured to accurately reflect the responsibilities, relationships, and flow of data within the system. It uses inheritance, composition, aggregation, and dependency arrows to clearly express how components collaborate in an ASP.NET Core MVC application. Controllers depend on Services and Repositories rather than on domain classes directly, which reduces coupling and supports dependency injection. Repositories encapsulate JSON-based persistence, so data access concerns never leak into controllers or views. The Service layer centralises session-based cart logic, keeping controllers lightweight and focused on request handling. Domain classes model real business concepts and their lifecycles: `Order` composes `OrderItem` (items cannot exist without an order), `ShoppingCart` aggregates modifiable `OrderItems`, `Customer` owns an `Address`, and `Category` aggregates multiple `Products`. View Models are intentionally separated to prevent views from binding directly to domain entities and to support strong validation. Together, the diagram demonstrates high cohesion within classes, clear boundaries between layers, and low coupling across the system, resulting in a maintainable, scalable, and testable architecture.

1.2. Justification of Changes and Non-Changes

1.2.1. Overall Changes/Non-Changes (at Class Level)

- **Adding Service layer**
 - The Service Layer encapsulates core business logic such as adding, updating, and clearing the cart, ensuring that the Controller remains focused on handling HTTP requests and responses. This approach prevents the Controller from becoming bloated with operations like updating quantities, managing cookies, or calculating totals, thereby upholding the Single Responsibility Principle (SRP). By clearly separating business logic from request handling, the architecture achieves better separation of concerns, improving maintainability, testability, and scalability while making the overall system easier to extend and manage.
- **Catalogue Class**
 - In Assignment 2, `Catalogue` directly managed a flat list of `Product` objects. Now, `Catalogue` no longer directly manages products - it now manages a collection of `Category` objects, and each `Category` in turn manages its own `Product(s)`.
 - Improves separation of concerns and better mirrors real-world catalogue structures, where a `Catalogue` organises categories rather than micromanaging every product. It also enhances extensibility, as new

categories can be added or removed without altering the core catalogue logic.

- **Category Class**

- Newly added class - introduced as a first-class entity.
- Represents logical groupings of products (e.g., Dairy, Snacks, Cleaning, Pantry, Drinks). This supports browsing, filtering and employee management of categories. cohesion by giving `Product` sole responsibility for product-specific data, while `Category` manages grouping and descriptive metadata. It also strengthens the domain model, aligning the design with customer experience and business operations.

- **Product Class**

- Unchanged in existence, but refined in role.
- Still represents individual items with attributes such as ID, name, price, stock, and description, but no longer carries responsibility for grouping (delegated to `Category`).

- **Payment Class**

- The `Payment` class remains in the design because it represents a core business concept essential to the checkout and order-finalisation workflow. Although Assignment 3 does not require real banking integration, the `Payment` class encapsulates credit-card validation logic (regex checks, Luhn algorithm, expiry validation), allowing the `Order` and `Invoice` workflows to remain realistic and modular. Keeping `Payment` separate maintains high cohesion: it handles only payment-related responsibilities, while `Order` and `Controller` classes stay focused on order creation and workflow management.

- **Customer Class**

- The `Customer` class remains mostly unchanged because it already fulfilled its core responsibilities: representing account details, storing contact information, and linking customer data to orders. In Assignment 3, it was refined with stricter validation rules (AU phone, postcode, address fields) but its role and collaborators remained identical: it still interacts with `Address`, `Order` history, and authentication workflows. Keeping `Customer` stable supports continuity between A2 and A3 while improving data quality.

- **Employee Class**

- The `Employee` class retains its conceptual role from Assignment 2: representing administrative users who have elevated privileges. In Assignment 3, the `Employee` class was extended only minimally with a clearer `Role = "Employee"` value to support ASP.NET Core Role-based Authorization. It collaborates with catalogue-related operations (e.g.,

managing products/categories) but remains deliberately simple because Assignment 3 does not require a full staff management subsystem.

- **Order Class**

- The `Order` class remains central and largely unchanged: it still contains order date, items, totals and customer information. A small change was introduced in Assignment 3 where the `Status` field is always set to "Successful" due to the simplified requirement (no real payment gateway). Other than that, the structure, composition relationship (`Order` composes many `OrderItem`), and flow remain the same.

- **OrderItem Class**

- `OrderItem` remains unchanged in responsibility and structure. It still represents an item within an order, containing product details, unit price, quantity and subtotal. Its collaborator relationships also remain the same (`Product` for pricing data, and `Order` or `ShoppingCart` for grouping). Keeping `OrderItem` unchanged made the transition from A2 to A3 seamless.

- **ShoppingCart Class**

- The `ShoppingCart` class remains unchanged in its purpose: an aggregate of `OrderItems` representing a temporary basket before purchase. The major modification was not to the class itself, but to how it is managed. In Assignment 3, the logic moved to the new `ShoppingCartService`. This means the `ShoppingCart` class stays simple and cohesive (just data), while all business logic is handled elsewhere.

- **Controller Classes**

- Controllers were significantly expanded and refined, but their core purpose did not change: they still handle HTTP requests and orchestrate workflows. Assignment 3 required more pages and interactions, so controllers were split into dedicated classes (e.g., `CatalogueController`, `ProductsController`, `OrdersController`). This increases cohesion and keeps each controller focused on one feature area. Their responsibilities remain aligned with MVC: receive requests, call services/repos, return views.

- **Repository Classes**

- Repository classes did not exist in Assignment 2, so they represent a major structural change. Assignment 3 required persistent JSON storage, and the Repository Pattern was introduced to support this requirement cleanly. Repositories take over responsibilities that were previously part of `Catalogue` or implicit in the design. Now they fully encapsulate file I/O,

identity generation, and data lookup, making controllers and services cleaner and more testable.

1.2.2. Changes/Non-Changes to Responsibilities and Collaborators

Non-Changes:

- **User Class:** The `User` class remained unchanged as it already provided a solid abstraction for shared user attributes and authentication logic. It continues to serve as the parent class for `Customer` and `Employee`, promoting reuse and consistency while upholding inheritance principles.
- **Customer Class:** The `Customer` class maintained its original responsibilities: managing account details, address, and order history.
- **Employee Class:** The `Employee` class retained its purpose of representing administrative users responsible for managing products and catalogue data.
- **Order Class:** The `Order` class remained central to recording customer purchases and linking to `OrderItem` and `Payment`. Its design was already robust and only required a small adjustment to fix the order status to "Successful" with no structural or relational changes.
- **OrderItem Class:** `OrderItem` continued to represent individual items within an order or cart, maintaining the same attributes (product, quantity, subtotal) and collaborators.
- **ShoppingCart Class:** The `ShoppingCart` class kept its purpose as a temporary collection of `OrderItems` before checkout. While cart logic was moved to a new `Service` layer, the class itself remained unchanged, preserving simplicity and cohesion as a data holder.

Changes:

Catalogue Class

Refer to [Table 1](#) below for the changes made to Catalogue class.

Table 1: Catalogue Class Comparison

Before (In Assignment 2)	Now	Justification
Knows the list of available Products. Collaborators: Product.	Knows list of available Categories. Collaborators: Product, Categories.	Categories were introduced in Assignment 3 to support structured browsing and filtering. Adding them

		ensures products can be grouped logically and improves navigation.
Supports browsing, filtering and searching of products. Collaborators: Product.	Supports browsing, filtering, searching of products <i>through categories</i> . Collaborators: Product, Categories.	Extending search and filter functionality to categories aligns with the new requirement for category-based browsing.
Provides product details to Customers. Collaborators: Customer, Product.	Provides category and product details to <i>Customers</i> . Collaborators: Product, Categories, Customer.	Customers now need access to both product and category information to complete their shopping journey.
Maintained and updated by Employees. Collaborators: Employee, Product.	Responsibilities stay the same. Collaborators: Employee, Categories, Product.	Responsibilities remain unchanged, but employees now also manage categories alongside products.
Keeps product listings current and accurate. Collaborators: Employee, Product.	Responsibilities stay the same. Collaborators: Employee, Categories, Product.	Responsibilities remain unchanged, extended to include categories to ensure catalogue accuracy.

Category Class

Refer to [Table 2](#) below for the changes made to Category class.

Table 2: Category Class Comparison

Before (In Assignment 2)	Now	Justification
N/A	Knows its own identifier, name and description Collaborators: Catalogue.	Introduced to uniquely define and describe categories, enabling structured catalogue management.
N/A	Maintain a list of <code>Product</code> objects that belong to the category. Collaborators: Product.	Required to group products under categories for browsing and filtering.
N/A	Provides access to products grouped under it.	Supports customer navigation by exposing products through

	Collaborators: Catalogue, Product.	category views.
N/A	Supports browsing and filtering of products by category Collaborators: Customer, Catalogue.	Added to meet Assignment 3 requirements for category-based browsing.
N/A	Can be created, updated or removed by <code>Employee</code> for catalogue management. Collaborators: Employee, Catalogue.	Ensures employees can maintain categories as part of catalogue administration.

Product Class

Refer to [Table 3](#) below for the changes made to Product class.

Table 3: Product Class Comparison

Before (In Assignment 2)	Now	Justification
Knows productId, SKU, name, description, price and stock quantity. Collaborators: N/A	Responsibilities stay the same. Collaborators: Category.	Responsibilities remain unchanged; now associated with categories for structured grouping.
Verifies that sufficient stock is available before confirming the order. Collaborators: OrderItem.	Responsibilities and collaborators stay the same.	Responsibilities remain unchanged; continues to ensure order validity.
Updates and decreases stock when an order is confirmed. Collaborators: Order.	Responsibilities and collaborators stay the same.	Responsibilities remain unchanged; maintains inventory accuracy.
Provides price information to other classes when requested. Collaborators: OrderItem.	Responsibilities and collaborators stay the same.	Responsibilities remain unchanged; supports order and checkout workflows
Allows administrators to update product details through the catalogue. Collaborators: Catalogue.	Responsibilities stay the same. Collaborators: Catalogue, Category.	Responsibilities remain unchanged; extended to include category association.
Makes sure product details are valid when a new	Responsibilities stay the same.	Responsibilities remain unchanged; validation now

product is added or updated. Collaborators: Catalogue.	Collaborators: Catalogue, Category.	considers category assignment as well.
--	--	--

Payment Class

Refer to [Table 4](#) below for the changes made to Payment class.

Table 4: Payment Class Comparison

Before (In Assignment 2)	Now	Justification
Recorded payment ID, amount, and status.	Now stores card details (CardNumber, Expiry, CVC) and OrderId.	The focus shifts from storing transaction outcomes to validating user payment input during checkout, better reflecting Assignment 3's functional design requirements.
Conceptually integrated with Stripe.	Stripe integration removed. Replaced with internal credit card validation logic.	Since Assignment 3 does not include real payment integration, the focus shifts to simulating payment behavior through validation logic (e.g., Luhn check, expiry). This allows testing and workflow realism without live banking APIs.
Credit card validation not specified.	Newly added responsibility.	Payment now includes input validation logic to ensure data accuracy and realism during checkout. This enhances modularity and allows Order and Controller classes to delegate payment checks cleanly.
Linked to one <code>Order</code> . Collaborators: Order.	Stays the same.	Payment remains associated with a single Order, preserving the 1-to-1 relationship between an order and its payment transaction.

Invalid Login Exception

Refer to [Table 5](#) below for the changes made.

Table 5: *InvalidLoginException* class comparison

Before (In Assignment 2)	Now	Justification
N/A	Newly introduced custom exception class <code>InvalidLoginException</code> .	Added to provide a clear, descriptive, and reusable way to handle invalid login attempts instead of using generic error messages.
N/A	Encapsulates login failure handling by throwing a specific exception with a predefined message ("Invalid email or password.").	Improves clarity, maintainability, and adherence to single responsibility. Controllers and services no longer need to hardcode repeated error strings.
N/A	Centralised error message through the exception class constructor.	Ensures consistency of feedback to users and simplifies localisation or future updates to login error handling.
N/A	Collaborators: Login/Authentication logic within <code>AccountController</code> or related service.	Clearly defines its role in authentication workflows, improving system robustness and readability.

1.2.3. Changes/Non-Changes to Dynamic Aspects (Bootstrap Process, Interaction Scenarios)

The dynamic aspects of the system, including the bootstrap process and interaction scenarios, were largely preserved from the design documented in Assignment 2. The following outlines the areas of continuity and the minor adjustments made, with justification:

- System Initialization
 - The application continues to initialize by loading essential product and employee data into a single `Catalogue` instance. In Assignment 3, this data is retrieved from JSON files rather than a database. This represents a change in the persistence mechanism but does not alter the dynamic flow of

initialization. The justification for this change is that JSON storage was explicitly required in Assignment 3, and it provides a lightweight alternative while maintaining the same logical behavior.

- Customer Registration
 - The registration process remains unchanged. A `Customer` instance is created upon registration, and a `ShoppingCart` is automatically associated. This mirrors the original design and required no modification, as the existing workflow already satisfied the requirements.
- Customer Checkout
 - No changes were made to the checkout process. Customers proceed from the shopping cart to checkout, where the order is finalized with product, quantity, and price details. This workflow was complete in Assignment 2 and continues to meet the requirements in Assignment 3. The justification for leaving this unchanged is that the process was already aligned with the expected customer journey.
- Order Processing
 - A minor change was introduced: the order status is always recorded as *Success*. This adjustment reflects Assignment 3 requirements, which do not require support for multiple payment options or failure scenarios. By simplifying order processing in this way, the implementation avoids unnecessary complexity while still fulfilling the assignment scope.
- Payment Confirmation and Clean-up
 - The dynamic behavior remains consistent with Assignment 2. Orders and payments are saved to persistent storage, and the `ShoppingCart` is cleared for future use. The only difference is that payment outcomes are fixed to “Success”, as justified above.

Overall, the bootstrap process and interaction scenarios were implemented as designed, with minimal changes. The only adjustments were the use of JSON files for persistence instead of a database, and the simplification of order status to always “Success” due to Assignment 3 simplification (cannot have banking system to validate transactions) . These changes are justified by the assignment scope and do not alter the fundamental dynamic behavior of the system. The customer journey, from system startup through registration, checkout, payment, and order history, behaves exactly as documented in Assignment 2, ensuring consistency and correctness.

2. Assignment 2 Design Discussion

2.1. Good Aspects of Assignment 2 Design (Original Design)

- Correct identification of domain classes: Catalogue, Product and Payment and their relationships.
- Recognition of the need for a single source of truth for the product listings (via Singleton).
- Clear mapping of business concepts (payment, product, catalogue) into the design.
- Correctly identified Model-View-Controller (MVC) as the architectural pattern, with clear separation of concerns.
- Naming conventions mentioned in Assignment 2 were still followed in Assignment 3.

2.2. Missing from Original Design

- No explicit modelling of Category as a first-class entity, which limited browsing or filtering functionality.
- A dedicated Service Layer was implemented to encapsulate business logic such as cart operations and data validation.
- Repository files were implemented to handle data storage in alignment with enterprise coding standards, ensuring clean separation between business logic.
- No mention of dependency injection or service lifetimes, which are central to ASP.NET Core MVC.
- Limited detail on partial views and reusable UI components.
- Payment class lacked internal validation (e.g., Luhn check, expiry date), which was added in Assignment 3.

2.3. Flawed Aspects of Original Design

- Over-reliance on a manual Singleton pattern for `Catalogue`, which could lead to tight coupling and reduced flexibility. In Assignment 3, this was replaced with ASP.NET Core's built-in dependency injection container using a registered singleton service.
- `Catalogue` class was overloaded, directly managing both products and responsibilities that should belong to `Categories`.
- No clear ownership of product-category relationship. The connection between products and categories was not clearly defined because there was no `Category` class to manage product grouping.

2.4. Level of Interpretation Required

- Some interpretation was needed to implement MVC in Assignment 3.
- Significant restructuring was needed to introduce repositories, services, and proper separation of concerns.
- The high-level class structure was sound, but significant refinement was needed:
 - Introducing `Category`, restructuring `Catalogue`.
 - The Singleton concept had to be re-interpreted in the context of ASP.NET Core's dependency injection container.
 - Introducing repositories and new controllers.
- The original design provided a strong foundation but required refinement to support real-world web application features.
- The manual Singleton pattern was replaced with scoped or singleton services registered in ASP.NET Core's dependency injection container.

3. Lessons Learnt

Throughout this project, the team learned the importance of defining scope and simplifications early to keep the design focused and avoid unnecessary features. Modelling the system around the user journey proved essential. For example, introducing the `Category` class improved browsing and filtering while keeping responsibilities clear. The team also realised how valuable it is to enforce strict separation between layers from the start: controllers should delegate to services, which then interact with repositories and domain models. Using dependency injection instead of manual singletons helped reduce coupling and improve testability. Implementing repositories early ensured data storage could change without affecting higher layers, while a shared validation specification kept rules like phone format, stock checks, and credit card validation consistent across the system. Finally, the team learned to apply design patterns selectively such as Service and Repository only where they genuinely simplify maintenance, and to always document the reasoning behind each class or decision to make the design easier to understand and extend in the future.

4. Architectural Styles

The grocery shopping web application adopts multiple architectural styles to ensure clarity, maintainability and alignment with best practices enterprise web application practices. The primary style is Layered Architecture, complemented by Model-View-Controller (MVC), the Repository Pattern and the inherent Client-Server Architecture of web systems. Together, these styles define the system's structure, connections and constraints.

4.1. Layered Architecture

The proposed grocery shopping web application adopts a Layered Architecture style. This architectural style divides the system into multiple layers - Presentation, Service, Data Access and Data - each with a distinct set of responsibilities and well-defined interactions with adjacent layers. The main purpose of this approach is to promote separation of concerns, modularity, and ease of maintenance, ensuring that changes in one part of the system can be made without affecting others. This layered approach is particularly suitable for enterprise web applications built using frameworks like ASP.NET Core, as it allows for clear abstraction between the user interface, business logic, and data management components.

The system is organized into four distinct layers:

- **Presentation Layer**
 - Controllers (`ProductsController`, `ShoppingCartController`) and Razor Views (`cshtml`) handle HTTP requests and present responses.
- **Service Layer**
 - Encapsulates business logic (e.g., `ShoppingCartService`) for operations such as adding products, updating quantities and validating availability.
- **Data Access Layer**
 - Interfaces (e.g., `ICatalogueRepository`) and implementations (e.g., `CatalogueRepository`) abstract data access, ensuring flexibility and testability.
- **Data Layer**
 - Domain models (e.g., `Catalogue`, `Category`, `Product`, `Order`, `Customer`) store product and category information, persisted in lightweight JSON files.

Connections follow a strict top-down flow: user requests enter at the Presentation Layer, cascade through Service and Data Access Layers, and reach the Data Layer. Results then flow back upward, maintaining separation of concerns.

4.2. Model-View-Controller (MVC)

Within the Layered Architecture, the MVC pattern operates primarily in the Presentation Layer, ensuring a structured flow between user interactions, application logic, and data models. Within the Presentation Layer, the system follows the MVC pattern:

- **Models** represent domain entities such as `Product`, `Category`, `Order`.
- **Views** (Razor templates) render dynamic HTML for product listings, shopping carts, checkout pages and others.
- **Controllers** process incoming requests, delegate to services and return responses.

This pattern enforces a clear separation between user interface, application logic and data representation, improving maintainability and testability.

4.3. Repository Pattern

The Repository Pattern is applied in the Data Access Layer. For instance, `CatalogueRepository` provides a consistent interface for retrieving and persisting product and category data stored in JSON files. This abstraction decouples business logic from data storage concerns, improving maintainability and facilitating future technology migrations (e.g., switching from JSON storage to a relational database) without impacting higher layers.

4.4. Client-Server Architecture

As a web application, the system inherently follows the Client-Server Architecture: the client (browser) sends HTTP requests, while the server (ASP.NET Core application running on IDE, macOS or Windows) processes requests, executes business logic and returns responses. This separation, based on stateless HTTP communication, ensures scalability, platform independence and secure client-server interactions.

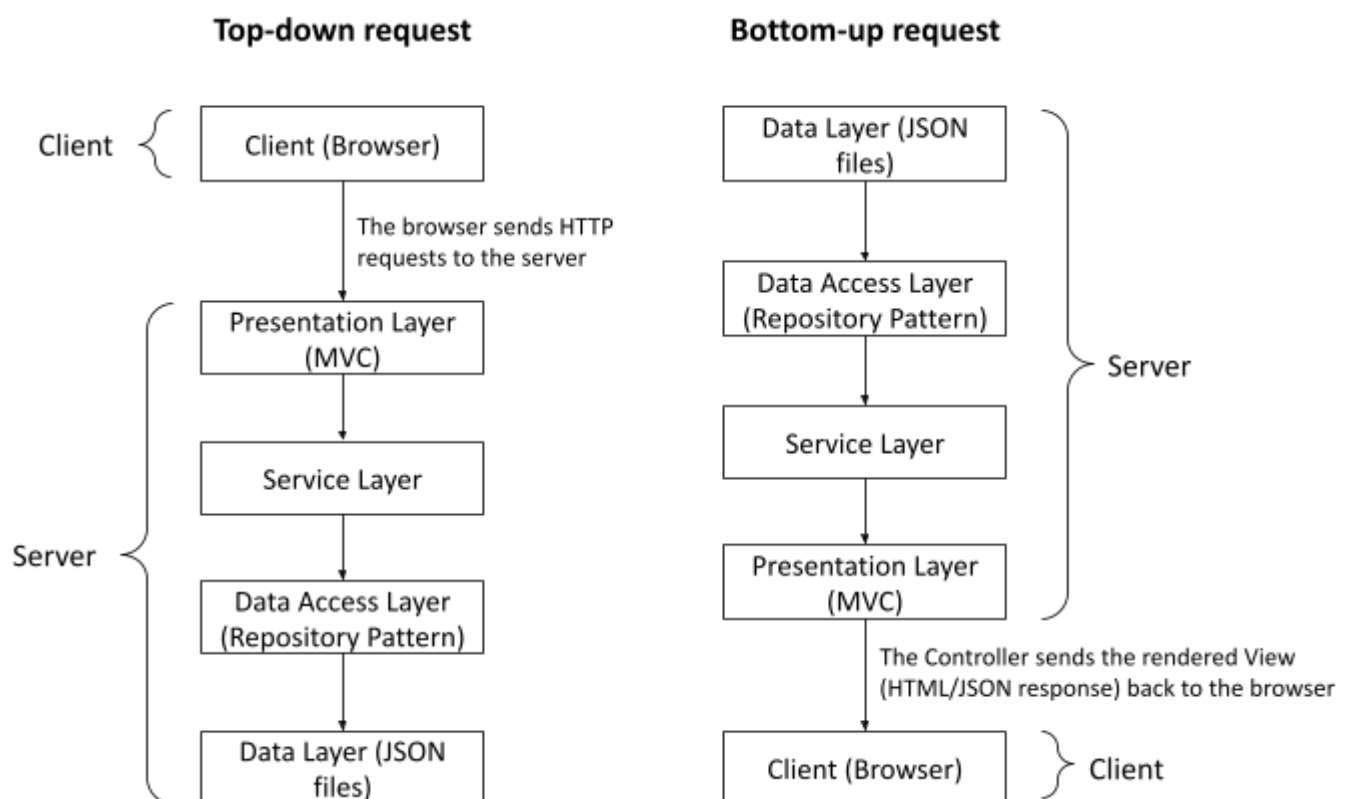
4.5. Constraints

While these architectural styles promote clarity and maintainability, they also introduce constraints. One of the constraints is performance overhead, as strict layering requires requests to pass through multiple layers, which can reduce efficiency. Additionally, cross-cutting concerns (e.g., logging, authentication or caching) can span multiple layers, making coordination and implementation more complex. Rigidity might also occur, since changes in lower layers (e.g., modifying the data storage format) can ripple upward and necessitate adjustments in higher layers. Finally, scalability limitations may arise, as layered architectures generally scale less effectively in distributed environments compared to styles like microservices or event-driven architectures.

4.6. Diagram Reference

[Figure 2](#) illustrates the system's layered architecture, showing how MVC and the Repository Pattern operate within a client-server environment. The top-down represents requests flowing from the client through the layers to the data, while the bottom-up represents responses flowing back to the client. The arrow from MVC to Client specifically represents the rendered response (HTML or JSON) returned to the browser.

Figure 2: Architectural Styles of the Your Local Shop Grocery Shopping Web Application



In [Figure 2](#) (Top-down request), at the top, the Client (Browser) is shown. It sends HTTP requests to the server; and in [Figure 2](#) (Bottom-up request), at the bottom, the Client (Browser) receives responses.

The Server (ASP.NET Core Application) is depicted as a layered stack:

- **Presentation Layer (MVC):** Controllers (`ProductsController`, `CartController`), Views (Razor templates), and Models (domain entities). Handles user interaction and request/response flow.
- **Service Layer:** Business logic components (e.g., `CartService`) validate stock, update carts, and calculate totals. Acts as the intermediary between controllers and data access.
- **Data Access Layer (Repository Pattern):** Interfaces (`ICatalogueRepository`) and implementations (`CatalogueRepository`) abstract persistence details.
- **Data Layer:** Domain models (`Catalogue`, `Category`, `Product`, `Order`, `Customer`) stored in JSON files.

The diagram shown in [Figure 2](#) also illustrates the system's architectural constraints:

- Strict layering enforces separation of concerns.
- Repository abstraction allows flexibility in persistence.
- MVC ensures clear UI logic separation.
- Client-Server enables cross-platform access.

In summary, by combining Layered Architecture, MVC, the Repository Pattern and Client-Server principles, the grocery shopping web application achieves a well-structured, maintainable and scalable foundation aligned with modern enterprise development standards.

5. Implementation

5.1. Chosen Language, Coding Standard Followed and Data Storage

5.1.1. Chosen language

The system was implemented in C# using ASP.NET Core MVC. The user interface was implemented with Razor `.cshtml` views styled with Bootstrap.

5.1.2. Coding Standard Followed

The implementation adheres to established C# coding conventions to ensure readability, maintainability, and consistency across the codebase. The following standards were applied:

- PascalCase
 - Used for naming classes, interfaces, and methods (e.g., `ShoppingCartController`, `CatalogueRepository`, `PlaceOrder`). This aligns with Microsoft's .NET coding conventions. (Microsoft, 2025).
- camelCase
 - Used for local variables and method arguments (e.g., `productId`, `orderQuantity`). This is consistent with industry practice (GeeksforGeeks, 2025).
- Exception suffix
 - Exception classes end with the suffix "Exception" (e.g., `InvalidLoginException`), following recommended naming guidelines (Poorte, 2023).
- Use meaningful names
 - Identifiers clearly describe their purpose (e.g., `ValidateCreditCard` rather than `DoStuff`), improving code clarity (Microsoft, 2025).
- Consistent indentation and braces
 - Code blocks used for-space indentation and braces are placed on new lines, following .NET style guidelines (Microsoft, 2025).
- Namespace organization
 - Namespaces reflect folder structure and logical grouping (e.g., `YourLocalShop.Controllers`, `YourLocalShop.Models`) (Microsoft, 2025).

5.1.3. Data storage

The system uses JSON (JavaScript Object Notation) files for data storage. Product catalogues, customer details, and order records are persisted in JSON format, allowing the application to read and write structured data consistently throughout the workflow.

5.2 Platform Used for Development and Testing of the Implementations

5.2.1. Platforms Used

- Rider IDE: For coding, debugging, and running the application.
- Microsoft Edge: For testing the web application in a modern browser environment.
- Mozilla Firefox: For cross-browser compatibility testing.
- MacOS: Development and testing on Apple systems.
- Windows: Development and testing on Microsoft systems.
- GitHub version control system: For source code management and collaborative version tracking.

5.2.2. Deployment and Execution

The program is deployed as a web application. It can be executed either by running `dotnet run` in the terminal or by clicking the “Run” button in Rider IDE. Both methods compile the project and launch it in the browser, confirming that the implementation builds without errors and runs correctly.

The application is hosted on localhost (e.g., <http://localhost:5164>), allowing interaction with the system in a browser. This local deployment ensures that all features can be verified.

In addition, the full source code is maintained on GitHub, which serves as the version control and distribution platform for the project. Hosting the repository on GitHub counts as a form of deployment, as it enables the application to be cloned, built, and executed consistently across different environments. The GitHub repository is available at: <https://github.com/vzunguyen/YourLocalShop>.

5.2.3. Application Startup

The application was successfully executed using both the .NET CLI and Rider IDE. Running the command `dotnet run` in the terminal compiles the project and launches it in the browser, confirming that the implementation builds without errors and runs correctly. Likewise, clicking the “Run” button in Rider IDE performs the same build and execution process, demonstrating that the system compiles and executes as expected across different environments.

Please check the demo video (<https://youtu.be/0tME2lgoA44>), which demonstrates that the application compiles and runs successfully.

5.3. Four Areas of Business Operations Implemented

5.3.1. Create Customer Account, Login and Logout

Customers can either create a new account or log in with existing credentials. When creating an account, the system presents a registration form where the customer must provide personal details including first name, last name, email address, phone number, password (with confirmation), and address (unit number, street number, street name, city, state, postcode, and country). All fields are mandatory except for the unit number, which is optional. Validation rules are enforced at the point of entry: the email must be in a valid email format, the phone number must begin with “04” and be exactly ten digits long, the postcode must be exactly four digits, and the unit number (if provided) must contain only numeric characters. These checks ensure that customer data is consistent and usable for order processing. Once the form passes validation and is submitted, the system creates the account, stores the details, and automatically logs the customer in, redirecting them to their account dashboard.

For returning customers, the login process requires entering their registered email and password. The system validates the credentials and, if correct, grants access to the account dashboard where customers can view their details (names, phone numbers, email and address) and order history. Invalid credentials trigger an error message with the option to retry or reset the password.

At any time, customers can securely log out by selecting the logout option in the navigation menu. This action clears the session and redirects them to the home or login page, ensuring account security.

Please check the demo video (<https://youtu.be/1Q5XQMmNMfw>), which provides a clear demonstration of the implemented functionality in action.

5.3.2. Browse Products, Search Products and Filter Products by Category

Customers can begin browsing products either by navigating directly to the products page or by clicking the Start Browsing button from the home page. The products page displays a catalogue of items, showing each product's name, category, and price. At this level, product descriptions and stock quantities are not shown, keeping the view concise for quick browsing.

A search function is available to allow customers to locate specific products by entering keywords. In addition, customers can filter products by selecting a category such as Dairy, Pantry, Cleaning, Drink, or Snack. Selecting a category will refresh the product list to show only items that belong to the chosen category. Customers may also combine search and category filters to narrow down results further.

For more detailed information, each product includes a More Details button. Clicking this opens a product detail view that displays the product's name, category, price, description, and the quantity of stock available. This view also provides a button to return to the main products page, ensuring smooth navigation between browsing and detailed inspection.

Please check the demo video (<https://youtu.be/cZfsAu79AH8>), which provides a clear demonstration of the implemented functionality in action.

5.3.3. Shopping Cart: Add Products to Cart, Change Product Quantity in Cart and Delete Product from Cart.

Customers can add products to their shopping cart directly from the products page by clicking the *Add to Cart* button displayed with each item. The same option is available on the product details page, where an *Add to Cart* button allows customers to include the selected product while viewing its detailed information.

At any time, customers can access their shopping cart by clicking the *Shopping Cart* link located in the top-right corner of the page. The shopping cart page displays all items currently added, along with options to manage them. Customers can adjust the quantity of each product, remove individual items, or clear the entire cart if desired.

The shopping cart page also provides navigation options: a *Continue Shopping* button allows customers to return to browsing products, while a *Checkout* button enables them to proceed directly to the checkout process. These features ensure that customers can flexibly manage their selections before finalizing their purchase.

Please check the demo video (<https://youtu.be/sFWCKIsvF1o>), which provides a clear demonstration of the implemented shopping cart functionality in action.

5.3.4. Place Order: Checkout, Payment and Order History

After adding products to the shopping cart, customers can proceed to place an order by clicking the Checkout button. This directs them to the Checkout page, where recipient details (first name, last name, email, and phone number) and delivery address are displayed. These fields are automatically prefilled based on the customer's account information, but can be modified if necessary. Input validations are enforced in the same way as during account creation: all fields are required except for the unit number, the postcode must be four digits, the phone number must begin with "04" and be ten digits long, the unit number must be numeric if provided, and the email must follow a valid format.

The Checkout page also displays the items selected for purchase, including product name, price, quantity, subtotal, and the overall total. Customers confirm their order by clicking the Place Order button, which generates an invoice and redirects them to the Invoice page. The Invoice page shows the invoice ID, recipient details and address, order date, order ID, and a detailed list of items with price, quantity, subtotal, and the total amount due.

Within the Invoice page, a Pay section is provided for entering credit card details. Validation rules ensure that the card number is 15-16 digits, the expiry date is in the correct format and not expired, and the CVC is 3-4 digits. A Luhn check is also performed to verify the validity of the card number. If all validations pass, the customer clicks the Pay button to complete the payment.

Successful payment redirects the customer to the Receipt page, which displays the receipt ID, order ID, confirmation message indicating payment success, date, recipient details, delivery address, and the full order breakdown (items, quantities, subtotals, and total).

Finally, customers can view their completed orders by navigating to the Account page and selecting the Order History option. This section lists past orders, providing a record of purchases and payment confirmations.

Please check the demo video (<https://youtu.be/wZbBuABLJWY>), which provides a clear demonstration of the implemented checkout, payment, and order history functionality in action.

6. References

GeeksforGeeks. (2025, July 15). *C# Coding Standards*. GeeksforGeeks. Retrieved November 3, 2025, from <https://www.geeksforgeeks.org/c-sharp/c-sharp-coding-standards/>

Microsoft. (2025, January 18). *.NET Coding Conventions - C#*. Microsoft Learn. Retrieved November 3, 2025, from <https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/coding-style/coding-conventions>

Poorte, J. (2023, September 30). *C# Coding Standards and Best Practices*. dofactory. Retrieved November 3, 2025, from <https://www.dofactory.com/csharp-coding-standards>

7. Appendix

7.1. Appendix 2: Assignment 2 Link

Below is the OneDrive link for SWE30003 Assignment 2 - Group 5:

https://liveswinburne.edu.au-my.sharepoint.com/:b:/g/personal/104511548_student_swin_e_du_au/EcrGRUbggjJluNYE1RCQMVEBqGd3JtVhs9D56nIQwCOLsg?e=aHML6T